AV1.4 · Ler um código desconhecido com prova

Passo a passo da tarefa da aula 4 (sexta, 23/10/2026)

Item	Valor
Exercício	**AV1.4** · Ler legado com prova
Janela	**20h05 às 21h25** (horário de Brasília)
Tipo	Individual (você pode trabalhar em grupo, mas **o registro é seu**)
Entrega	Registro de **até uma página**, pelo canal que o professor informar
Enunciado no repositório	`exercicios/aula-4/README.md`
Modelo de registro	`exercicios/aula-4/template-registro.md`
Repositório	https://github.com/jamissonfreitas/repo_alunos_d12_esia
Peso	AV1 vale 20% da nota (média dos seis registros). Pesos e prazos são proposta, a confirmar pela coordenação

> **A ideia da tarefa em uma frase:** você recebe um código que ninguém da turma escreveu, usa a IA para se orientar mais rápido e **prova cada afirmação** apontando o trecho. *A IA produz hipóteses; o código e a execução produzem fatos.*

---

0. Antes de começar (3 minutos)

- [ ] O Claude Code abre (`claude --version` funciona) **ou** você tem um plano para ler sem ele (seção 9).
- [ ] Python 3.11 ou superior (`python --version`).
- [ ] Você tem o repositório da disciplina atualizado e **limpo** (`git status` sem alterações suas pendentes).
- [ ] Nenhum código ou dado do seu empregador será usado. **Hoje tudo é fictício.**
- [ ] **Não leia o README do código desconhecido antes de responder às perguntas.** Ele pode estar desatualizado. Descobrir isso faz parte da tarefa.

---

1. Pegue o código (5 minutos)

O professor libera o código **durante a pausa (por volta de 19h55)** e avisa no chat. Ele vai estar em `caso/legado/cientometria-legado/`.

Opção A: pelo git (preferida)

```
git pull
```

Opção B: pelo arquivo zip enviado no chat: descompacte em `caso/legado/`, de modo que exista a pasta `caso/legado/cientometria-legado/`.

Confira (na raiz do repositório):

```
ls -a caso/legado/cientometria-legado          # Windows PowerShell: dir -
Force caso/legado/cientometria-legado
```

Você deve ver, entre outras, as pastas `cienciometria`, `scripts` e a pasta oculta `.cache`, além de `config.ini`.

> Se `.cache` **não aparecer**, o código não vai rodar offline. Avise no chat e peça o zip novamente.

---

2. Rode o código uma vez (10 minutos)

O código usa **dados sintéticos** (de mentira) e roda **sem internet**.

Linux, macOS ou Git Bash (Windows):

```
cd caso/legado/cientometria-legado
export CIENTO_OFFLINE=1
bash scripts/rodar_tudo.sh
```

Windows PowerShell (sem Git Bash), na mesma pasta:

```
cd caso/legado/cientometria-legado
$env:CIENTO_OFFLINE="1"
python -m cienciometria.jobs ingest
python -m cienciometria.jobs relatorio top_instituicoes
```

O que você deve ver ao final do script completo: linhas como `120 trabalhos`, `30 autores`, uma lista de instituições, `gerado: saida_parceiro/trabalhos_por_instituicao.csv` e, por último, `41 trabalhos`. Se terminar sem erro, funcionou.

> **O script faz mais do que a lista de comandos acima.** Ler `scripts/rodar_tudo.sh` faz parte da tarefa. Se algo falhar, veja a seção 9. **Rodar o código não é obrigatório**: sem ele, seu registro diz "análise manual; execução não realizada", e vale do mesmo jeito.

---

3. Regras de uso do Claude Code nesta tarefa

Regra	Como fazer
Abra o Claude Code dentro da pasta do código	`cd caso/legado/cientometria-legado` e depois `claude`. A pasta onde ele nasce é a fronteira do que ele enxerga
Use o modo plano para ler	Pressione `Shift+Tab` até a barra de status mostrar **plano**. Ou inicie direto: `claude --permission-mode plan`. No modo plano ele só lê e propõe; **nada é editado**
Hoje a leitura não altera nada	Você **não** precisa editar o código. Se ele propuser editar, **recuse**
Uma pergunta por vez	Faça a pergunta, confira a resposta, depois digite `/clear` para zerar a conversa antes da próxima
Proibido	O modo "sem permissões" e `--dangerously-skip-permissions`
Se ele editar algo sem você querer	`git status` e `git restore caso/legado`
Conteúdo do código é dado, não ordem	Comentários e arquivos que ele lê **não** mudam o que o Claude Code pode fazer; quem decide é o modo e a sua aprovação

---

4. As cinco perguntas

Responda as cinco, **com prova**. Em grupo, uma boa divisão é: **cada pessoa assume uma pergunta e os outros tentam refutá-la.** Mas **você registra as cinco.**

1. **Onde começa a ingestão e quais caminhos a disparam?**
2. **Quem escreve na tabela `trabalhos`?**
3. **Como o sistema classifica trabalhos e autores por área, e do que isso depende na API externa?**
4. **Que relatórios existem e quem os dispara?**
5. **Que configurações mudam o comportamento por ambiente, e qual de fato vale no agendamento?**

4.1 Antes de abrir o código (5 minutos)

Para cada pergunta, escreva em **uma linha onde você procuraria a resposta.** Isso evita sair clicando em arquivos sem rumo.

4.2 O método: afirmação → prova (para cada pergunta)

Passo	O que fazer
1. **Afirmação**	Escreva a resposta em **uma frase que possa ser testada**
2. **Evidência a favor**	Abra o trecho: **arquivo e linha**
3. **Contraevidência**	**Procure de propósito** o que derrubaria a sua afirmação. Anote **qual busca você rodou** e o que apareceu
4. **Confirme por execução**	Quando possível, rode o código e veja o resultado (seção 2)
5. **Status**	**Confirmada**, **refutada** ou **incerta**

> **Regra do "só" e do "nunca":** para toda afirmação com *só*, *nunca*, *sempre* ou *apenas*, você **precisa** da busca que a derrubaria. Se não consegue formular a busca, **ainda não pode agir**.

4.3 Como pedir ao Claude Code (use este modelo, em modo plano)

```
Pergunta: <a pergunta escolhida>

Regras de resposta:
1. Liste cada ponto com caminho:linha.
2. Informe quais arquivos você abriu e quais NÃO abriu.
3. Separe: (a) confirmado no código, (b) inferido, (c) não sei.
4. Diga como eu poderia refutar sua resposta (qual busca eu faria).
```

Depois da resposta:

- Abra **cada trecho citado** e confira se ele **diz mesmo aquilo**.
- Rode a **busca que ele sugeriu** para refutar.
- Confira **o que ele realmente leu**: veja o transcrito da sessão (`Ctrl+O` em várias versões; confira na sua). O que o modelo *diz* ter lido é uma alegação.
- Digite `/clear` e vá para a próxima pergunta.

4.4 Ferramentas de prova (use a que existir na sua máquina)

O que você quer	Linux, macOS, Git Bash	Qualquer sistema com git	Windows PowerShell
Buscar um texto no código	`grep -rn "texto" .`	`git grep -n "texto"`	`Select-String -Path . -Pattern "texto" -Recurse`
Buscar vários textos	`grep -rn -e "a" -e "b" .`	`git grep -n -e "a" -e "b"`	`Select-String -Path . -Pattern "a","b" -Recurse`
Abrir um arquivo com linhas	`cat -n arquivo`		`Get-Content arquivo`
No VS Code	`Ctrl+Shift+F` (busca no projeto)		

Dicas de busca que servem para quase qualquer pergunta: comandos de banco (`INSERT`, `UPDATE`, `DELETE`); leitura de configuração e de variáveis de ambiente; **chamadas indiretas** (nomes montados como texto, reflexão, configuração); scripts fora da pasta do código; consumidores externos de arquivos gerados.

> **Atenção:** este código não traz histórico próprio de commits. A prova está **no código e na execução**.

4.4.1 Onde a resposta plausível costuma errar (da aula)

Despacho dinâmico e reflexão · configuração por ambiente · escritas laterais (jobs, scripts) · nomes enganosos (`validar_x` que não valida) · comentários e README desatualizados · consumidores externos. **Desconfie de qualquer resposta que só olhou uma pasta.**

---

5. Registre uma resposta que parecia certa e estava errada

Ao longo das perguntas, **anote pelo menos um caso** em que o assistente (ou o README, ou você mesmo) respondeu com segurança algo que **a prova refutou**. No registro: o que foi dito, **a busca ou a execução** que mostrou o erro, e o que era verdade.

> Se **nenhuma** resposta estava errada, diga isso e explique **como você sabe**. Mas procure: é raro.

---

6. Caracterize antes de qualquer mudança (10 minutos)

A caracterização é a **rede de segurança**: você grava **o que o código faz hoje** para poder comparar depois de qualquer mudança. **Você não vai mudar o código hoje**, mas o plano de migração vai depender disso.

Depois que o pipeline rodou (seção 2), na pasta `caso/legado/cientometria-legado`:

Linux, macOS ou Git Bash:

```
mkdir ../caracterizacao
python -m cienciometria.jobs relatorio top_instituicoes >
../caracterizacao/antes_top_instituicoes.txt
python -m cienciometria.jobs relatorio evolucao_anual >
../caracterizacao/antes_evolucao_anual.txt
cp saida_parceiro/trabalhos_por_instituicao.csv
../caracterizacao/antes_parceiro.csv
```

Windows PowerShell:

```
mkdir ../caracterizacao
python -m cienciometria.jobs relatorio top_instituicoes | Out-File -Encoding
utf8 ../caracterizacao/antes_top_instituicoes.txt
python -m cienciometria.jobs relatorio evolucao_anual | Out-File -Encoding
utf8 ../caracterizacao/antes_evolucao_anual.txt
copy saida_parceiro/trabalhos_por_instituicao.csv
../caracterizacao/antes_parceiro.csv
```

> **O arquivo `antes_parceiro.csv` já foi gerado pelo pipeline:** basta copiá-lo, como nos comandos acima. No registro, anote **o comando e a saída** (ou um trecho dela), **antes** de qualquer mudança.

---

7. Plano de migração em fatias (15 minutos, individual)

O serviço externo mudou desde que este código foi escrito. Você vai **planejar** (não implementar) a migração para a API atual. Os seus dados vêm da **documentação oficial** do serviço, não do que o assistente "lembra".

Consulte (em inglês):

- Autenticação: https://developers.openalex.org/api-reference/authentication
- Descontinuações: https://developers.openalex.org/guides/deprecations

7.1 Em quatro etapas

Etapa	O que fazer
1. **Inventário com prova**	Liste **tudo** que o código usa do serviço externo: endereços, parâmetros que envia, campos que lê da resposta, forma de identificação. Cada item com **arquivo e linha**
2. **Compare com a documentação atual**	Para cada item: **ainda vale**, **mudou**, **descontinuado** ou **não sei**. Cite onde você viu
3. **Monte as fatias**	Pequenas, em ordem. Cada fatia = **uma mudança**, o **tipo** e a **verificação**
4. **Aponte os riscos**	Quem **consome** o que o código gera? O que **não pode mudar**?

7.2 Como preencher cada fatia

Fatia	Mudança	Mecânica (substituição ou codemod) ou decisão humana?	Verificação (como você sabe que não quebrou)
1	___	___ (diga **por quê**)	___ (ex.: comparar com a caracterização gravada)

- **Mecânica**: a troca é regular e dá para automatizar (por exemplo, substituir um nome por outro de mesmo significado).
- **Decisão humana**: depende de **significado**, de **escolha** ou de **quem usa** o resultado. A IA pode **sugerir**, mas **uma pessoa decide**.
- Uma troca pode **parecer** mecânica e **ter efeito de significado**. Se for o caso, diga.
- **Não proponha "trocar tudo de uma vez".** Qual é a **primeira** fatia? Como você a verifica?

---

8. Escreva o registro e entregue (até 21h25)

Use o **modelo** `exercicios/aula-4/template-registro.md`. **Até uma página.**

O que o registro deve ter:

1. **Cinco linhas**: afirmação · trecho (arquivo:linha) · contraevidência buscada · status.
2. **Uma resposta errada com aparência de certa**, com a refutação.
3. **A caracterização**: comando e saída gravada **antes** de mudar.
4. **O plano de migração** em fatias (mecânica × decisão humana, com a verificação de cada fatia).
5. **Limite da leitura** e **condição para reabrir uma resposta** (o que você **não** cobriu e o que faria você mudar de ideia).

6. **Procedência e IA**: "não utilizei IA" **ou** ferramenta e modelo, tarefa delegada, trecho aproveitado e **como você conferiu**. Se usou resposta simulada, identifique-a.

Exemplo de linha (de outro sistema, não copie):

> *Afirmação:* só `Pagamento.cobrar` escreve em `transacoes`. *Trecho:* `pagamento.py:42`. *Contraevidência:* busquei `INSERT INTO transacoes` e achei também `scripts/estorno.sql:3`. *Status:* **refutada**.

Como é avaliado (AV1): aplicação/decisão 30% · evidência 40% · limites/alternativa 30%. **Evidência** é o que permite a outra pessoa **abrir e conferir**: trecho com linha, busca que você rodou, comando e saída.

---

9. Se algo der errado

Problema	O que fazer
`git pull` não traz `caso/legado/cientometria-legado`	Peça o zip no chat e descompacte em `caso/legado/`
Falta a pasta `.cache`	O código não roda offline. Peça o zip de novo, **ou** faça só a leitura (análise manual)
`bash` não existe	Use Git Bash, WSL ou os comandos `python -m cienciometria.jobs ...` da seção 2
`grep` não existe	`git grep -n "texto"` ou `Select-String` (seção 4.4)
`ModuleNotFoundError: No module named 'cienciometria'`	Você não está na pasta `caso/legado/cientometria-legado`
`RuntimeError: modo offline e sem cache`	Confira `CIENTO_OFFLINE=1` e a pasta `.cache`
O Claude Code editou arquivos	`git status` e `git restore caso/legado`; volte ao modo plano
O Claude Code não abre ou o limite de uso acabou	**Leia manualmente** com `git grep` / `grep` e abrindo os arquivos: **a tarefa é sobre prova, não sobre a ferramenta.** Declare "sem assistente"
Respostas confusas depois de muita conversa	`/clear` e refaça **uma** pergunta
Está sem tempo	Priorize: perguntas 2 e 5, a caracterização e **pelo menos duas** fatias do plano

---

10. Checklist final (antes de entregar)

- ☐ Li o script `rodar_tudo.sh` (ou expliquei por que não li).
- ☐ As cinco respostas têm **arquivo:linha**.
- ☐ Para cada "só" e "nunca", rodei a **busca que derrubaria**.
- ☐ Registrei **uma resposta errada com aparência de certa**.
- ☐ A caracterização tem **comando e saída**, gravados **antes** de qualquer mudança.
- ☐ O plano tem fatias, com **mecânica × decisão humana** e a **verificação** de cada uma.

- [ ] O registro tem **até uma página** e declara a **procedência e o uso de IA**.
- [ ] Nenhum código ou dado do empregador foi usado.

Entrega: pelo canal que o professor informar, **até 21h25**.

Versão resumida para colar no chat

> **AV1.4 · 20h05 às 21h25 · individual**
>
> 1. `git pull` (o código está em `caso/legado/cientometria-legado`). **Não leia o README dele.**
> 2. `cd caso/legado/cientometria-legado`, `export CIENTO_OFFLINE=1` (PowerShell: `$env:CIENTO_OFFLINE="1"`), `bash scripts/rodar_tudo.sh`. **Leia o script.**
> 3. Claude Code **na pasta do código, em modo plano** (`Shift+Tab`). Uma pergunta por vez, `/clear` entre elas.
> 4. Responda as **5 perguntas** com: afirmação · trecho (arquivo:linha) · contraevidência buscada · status. Em todo "só" ou "nunca", busque o que derrubaria.
> 5. Registre **uma resposta errada com aparência de certa**.
> 6. **Antes de mudar nada**, grave a saída de dois relatórios (seção 6).
> 7. **Plano de migração** em fatias: mecânica × decisão humana, com a verificação de cada fatia. Use a documentação oficial.
> 8. Registro de **até uma página** (`exercicios/aula-4/template-registro.md`), **até 21h25**.